

# Complexity and Integer Linear Programming

9/14/26

Mihir Tandon

September 14, 2026



## Announcements

- We are finalizing exact projects for the algorithm/development teams, these will be github issues in the SIGma repo.
- Please see the SIGma CG:SHOP repo for further details, and make sure you filled out the spreadsheet in the discord if you are interested!
- We have a solution checker implemented now!



# Outline

Big O Notation

Linear Programming

Integer Linear Programming



# Section 1

## Big O Notation



## How many computations does this do?

```
Algorithm Sum(A, n):  
    total = 0  
    for i = 1 to n  
        total = total + A[i]  
    return total
```

- How many additions does this algorithm perform on an array of  $n$  values?



## How many computations does this do?

```
Algorithm Sum(A, n):  
    total = 0  
    for i = 1 to n  
        total = total + A[i]  
    return total
```

- How many additions does this algorithm perform on an array of  $n$  values?
- **Answer:** Exactly  $n$  additions. Doubling the input doubles this count.



## What changes with two loops?

```
Algorithm ComparePairs(A, n):  
    for i = 1 to n  
        for j = i + 1 to n  
            compare(A[i], A[j])
```

- How many comparisons does this algorithm perform?



## What changes with two loops?

```
Algorithm ComparePairs(A, n):  
    for i = 1 to n  
        for j = i + 1 to n  
            compare(A[i], A[j])
```

- How many comparisons does this algorithm perform?
- **Answer:**  $(n - 1) + (n - 2) + \dots + 1 = \frac{n(n-1)}{2}$  comparisons.



## What changes with two loops?

```
Algorithm ComparePairs(A, n):  
    for i = 1 to n  
        for j = i + 1 to n  
            compare(A[i], A[j])
```

- How many comparisons does this algorithm perform?
- **Answer:**  $(n - 1) + (n - 2) + \dots + 1 = \frac{n(n-1)}{2}$  comparisons.
- For large  $n$ , doubling the input roughly quadruples this count.



## What if the input keeps shrinking?

```
Algorithm Halve(n):  
    while n > 1  
        n = floor(n / 2)
```

- For an integer  $n \geq 1$ , how many times does the loop divide by two?



## What if the input keeps shrinking?

```
Algorithm Halve(n):  
    while n > 1  
        n = floor(n / 2)
```

- For an integer  $n \geq 1$ , how many times does the loop divide by two?
- **Answer:**  $\lfloor \log_2 n \rfloor$  times. For  $n = 16$ , the values are 16, 8, 4, 2, 1, so there are four divisions.



## What if the input keeps shrinking?

```
Algorithm Halve(n) :  
    while n > 1  
        n = floor(n / 2)
```

- For an integer  $n \geq 1$ , how many times does the loop divide by two?
- **Answer:**  $\lfloor \log_2 n \rfloor$  times. For  $n = 16$ , the values are 16, 8, 4, 2, 1, so there are four divisions.
- Let  $T(n)$  count the number of operations. Big O describes how that count grows with the input size.



## Big O notation

- We write  $T(n) = O(g(n))$  if constants  $c > 0$  and  $n_0$  exist such that

$$0 \leq T(n) \leq c g(n) \quad \text{for all } n \geq n_0.$$



## Big O notation

- We write  $T(n) = O(g(n))$  if constants  $c > 0$  and  $n_0$  exist such that

$$0 \leq T(n) \leq c g(n) \quad \text{for all } n \geq n_0.$$

- The constant  $c$  allows a fixed multiplicative difference, and  $n_0$  marks where the bound begins to hold.



## Big O notation

- We write  $T(n) = O(g(n))$  if constants  $c > 0$  and  $n_0$  exist such that

$$0 \leq T(n) \leq c g(n) \quad \text{for all } n \geq n_0.$$

- The constant  $c$  allows a fixed multiplicative difference, and  $n_0$  marks where the bound begins to hold.
- For example,  $3n + 7 \leq 4n$  whenever  $n \geq 7$ . Therefore,  $3n + 7 = O(n)$ .



## Big O notation

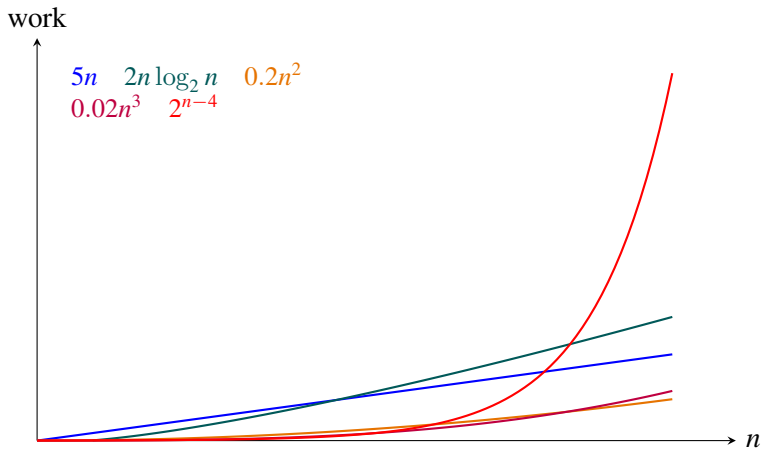
- We write  $T(n) = O(g(n))$  if constants  $c > 0$  and  $n_0$  exist such that

$$0 \leq T(n) \leq c g(n) \quad \text{for all } n \geq n_0.$$

- The constant  $c$  allows a fixed multiplicative difference, and  $n_0$  marks where the bound begins to hold.
- For example,  $3n + 7 \leq 4n$  whenever  $n \geq 7$ . Therefore,  $3n + 7 = O(n)$ .
- Big O gives an upper bound. A linear runtime is also  $O(n^2)$ , although that bound is less useful.



# Growth functions



## A convex hull recurrence

- Consider a divide-and-conquer convex hull algorithm on  $n$  points sorted by  $x$ -coordinate.



## A convex hull recurrence

- Consider a divide-and-conquer convex hull algorithm on  $n$  points sorted by  $x$ -coordinate.

```
Hull(P) :  
    if |P| <= 3: return the hull of P  
    split P into left and right halves  
    H_left  = Hull(left)  
    H_right = Hull(right)  
    return MergeHulls(H_left, H_right)
```



## A convex hull recurrence

- Consider a divide-and-conquer convex hull algorithm on  $n$  points sorted by  $x$ -coordinate.

```
Hull(P) :  
    if |P| <= 3: return the hull of P  
    split P into left and right halves  
    H_left  = Hull(left)  
    H_right = Hull(right)  
    return MergeHulls(H_left, H_right)
```

- The recursive calls solve two subproblems of size  $n/2$ . The merge takes  $O(n)$  time.



## A convex hull recurrence

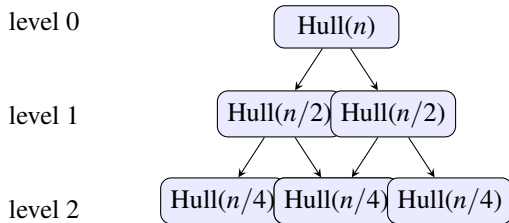
- Consider a divide-and-conquer convex hull algorithm on  $n$  points sorted by  $x$ -coordinate.

```
Hull(P) :  
    if |P| <= 3: return the hull of P  
    split P into left and right halves  
    H_left = Hull(left)  
    H_right = Hull(right)  
    return MergeHulls(H_left, H_right)
```

- The recursive calls solve two subproblems of size  $n/2$ . The merge takes  $O(n)$  time.
- This gives the recurrence  $T(n) = 2T(n/2) + O(n)$ .



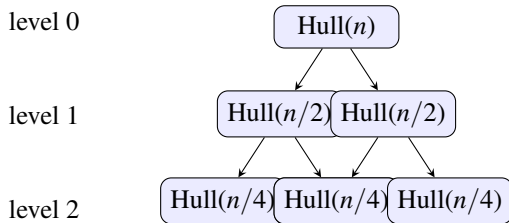
## The recursion tree



- At level 0, one merge handles  $n$  boundary edges.



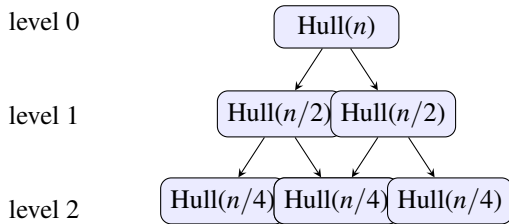
## The recursion tree



- At level 0, one merge handles  $n$  boundary edges.
- At level 1, two merges each handle at most  $n/2$  edges, for at most  $n$  total.



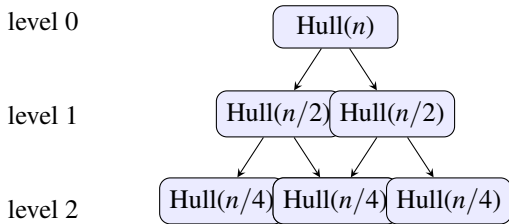
## The recursion tree



- At level 0, one merge handles  $n$  boundary edges.
- At level 1, two merges each handle at most  $n/2$  edges, for at most  $n$  total.
- Each lower level also performs at most  $O(n)$  total merge work. There are  $O(\log n)$  levels until subproblems have at most three points.



## The recursion tree



- At level 0, one merge handles  $n$  boundary edges.
- At level 1, two merges each handle at most  $n/2$  edges, for at most  $n$  total.
- Each lower level also performs at most  $O(n)$  total merge work. There are  $O(\log n)$  levels until subproblems have at most three points.
- Therefore  $T(n) = O(n \log n)$ .



## Section 2

### Linear Programming



## A production problem

- A workshop makes tables and chairs. It has 7 planks of wood and enough time to assemble 5 items.



## A production problem

- A workshop makes tables and chairs. It has 7 planks of wood and enough time to assemble 5 items.

|               | Table     | Chair     |
|---------------|-----------|-----------|
| Wood required | 2 planks  | 1 plank   |
| Profit        | 3 dollars | 2 dollars |



## A production problem

- A workshop makes tables and chairs. It has 7 planks of wood and enough time to assemble 5 items.

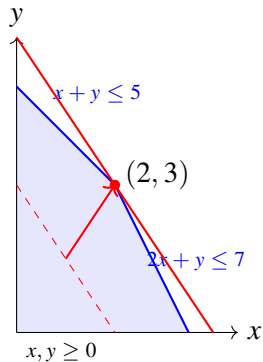
|               | Table     | Chair     |
|---------------|-----------|-----------|
| Wood required | 2 planks  | 1 plank   |
| Profit        | 3 dollars | 2 dollars |

- Let  $x$  count tables and  $y$  count chairs. Ignoring whole-number restrictions for now, the LP is

$$\begin{array}{ll}\max & 3x + 2y \\ \text{subject to} & 2x + y \leq 7, \quad x + y \leq 5, \\ & x, y \in \mathbb{R}_{\geq 0}.\end{array}$$



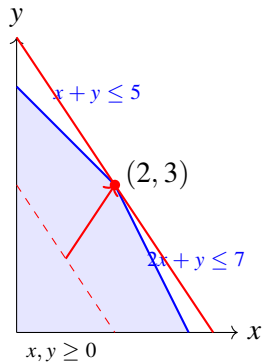
## The geometry of the production LP



- Each inequality defines a half-plane in two dimensions, or a halfspace in  $d$  dimensions. Their intersection is the **feasible region**.



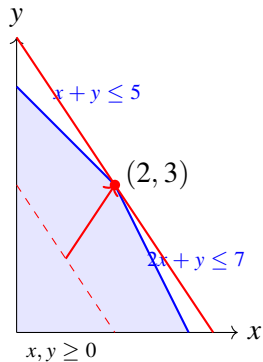
## The geometry of the production LP



- Each inequality defines a half-plane in two dimensions, or a halfspace in  $d$  dimensions. Their intersection is the **feasible region**.
- Here,  $x, y \geq 0$ ,  $2x + y \leq 7$ , and  $x + y \leq 5$  define the shaded region.



## The geometry of the production LP



- Each inequality defines a half-plane in two dimensions, or a halfspace in  $d$  dimensions. Their intersection is the **feasible region**.
- Here,  $x, y \geq 0$ ,  $2x + y \leq 7$ , and  $x + y \leq 5$  define the shaded region.
- To maximize  $3x + 2y$ , move the line  $3x + 2y = t$  toward larger  $t$ . The last contact occurs at  $(2, 3)$ , with value 12.



## The pieces of a two-dimensional LP

- A **half-plane constraint** has the form

$$ax + by \leq r.$$

Its boundary is the line  $ax + by = r$ , and the inequality keeps one side of that line.



## The pieces of a two-dimensional LP

- A **half-plane constraint** has the form

$$ax + by \leq r.$$

Its boundary is the line  $ax + by = r$ , and the inequality keeps one side of that line.

- The intersection of all half-planes is the **feasible region**. These are the points that satisfy every constraint.



## The pieces of a two-dimensional LP

- A **half-plane constraint** has the form

$$ax + by \leq r.$$

Its boundary is the line  $ax + by = r$ , and the inequality keeps one side of that line.

- The intersection of all half-planes is the **feasible region**. These are the points that satisfy every constraint.
- An **objective vector**  $c = (c_x, c_y)$  assigns a score

$$c \cdot (x, y) = c_x x + c_y y.$$

For example, maximizing  $3x + 2y$  uses objective vector  $(3, 2)$ .



## The pieces of a two-dimensional LP

- A **half-plane constraint** has the form

$$ax + by \leq r.$$

Its boundary is the line  $ax + by = r$ , and the inequality keeps one side of that line.

- The intersection of all half-planes is the **feasible region**. These are the points that satisfy every constraint.
- An **objective vector**  $c = (c_x, c_y)$  assigns a score

$$c \cdot (x, y) = c_x x + c_y y.$$

For example, maximizing  $3x + 2y$  uses objective vector  $(3, 2)$ .

- A linear program chooses a feasible point with the largest or smallest score.



## Input and output of Seidel's LP algorithm

- **Seidel's algorithm [1992]** solves a bounded two-dimensional LP by adding constraints one at a time in random order.



## Input and output of Seidel's LP algorithm

- **Seidel's algorithm [1992]** solves a bounded two-dimensional LP by adding constraints one at a time in random order.
- **Input:** half-plane constraints  $H = \{h_1, \dots, h_n\}$ , an objective direction  $c$ , and a bounding box  $B$  containing the feasible region. Each constraint keeps one side of a line (a two-dimensional halfspace).



## Input and output of Seidel's LP algorithm

- **Seidel's algorithm [1992]** solves a bounded two-dimensional LP by adding constraints one at a time in random order.
- **Input:** half-plane constraints  $H = \{h_1, \dots, h_n\}$ , an objective direction  $c$ , and a bounding box  $B$  containing the feasible region. Each constraint keeps one side of a line (a two-dimensional halfspace).
- **Goal:** find an allowed point  $(x, y)$  that maximizes a linear score such as  $3x + 2y$ .



## Input and output of Seidel's LP algorithm

- **Seidel's algorithm [1992]** solves a bounded two-dimensional LP by adding constraints one at a time in random order.
- **Input:** half-plane constraints  $H = \{h_1, \dots, h_n\}$ , an objective direction  $c$ , and a bounding box  $B$  containing the feasible region. Each constraint keeps one side of a line (a two-dimensional halfspace).
- **Goal:** find an allowed point  $(x, y)$  that maximizes a linear score such as  $3x + 2y$ .
- **Output:** the best point, or a report that no allowed point exists.



## Seidel's two-dimensional algorithm

```
Seidel2D(H, c, B):  
    randomly order H as h_1, ..., h_n  
    p = corner of B maximizing c dot p  
    for i = 1 to n:  
        if p violates constraint h_i:  
            p = BestOnLine(h_i, H[1..i-1], B, c)  
            if p is infeasible: return infeasible  
    return p  
  
BestOnLine(h, P, B, c):  
    I = boundary(h) intersect B  
    for g in P: I = I intersect g  
    if I is empty: return infeasible  
    return the endpoint of I with the best score
```



## Seidel's two-dimensional algorithm

```
Seidel2D(H, c, B):  
    randomly order H as h_1, ..., h_n  
    p = corner of B maximizing c dot p  
    for i = 1 to n:  
        if p violates constraint h_i:  
            p = BestOnLine(h_i, H[1..i-1], B, c)  
            if p is infeasible: return infeasible  
    return p  
  
BestOnLine(h, P, B, c):  
    I = boundary(h) intersect B  
    for g in P: I = I intersect g  
    if I is empty: return infeasible  
    return the endpoint of I with the best score
```

- BestOnLine intersects the new boundary with the earlier constraints and returns its best endpoint.



## A two-dimensional linear program

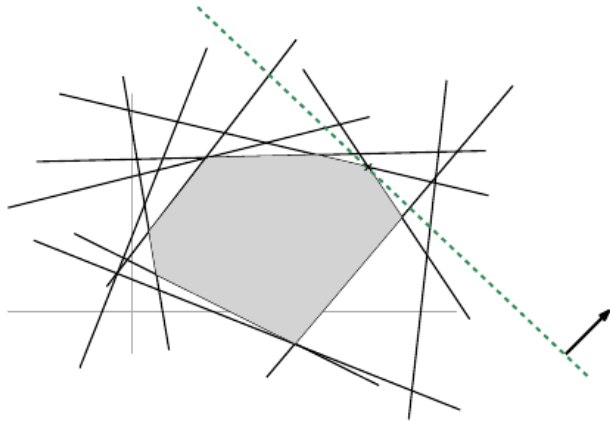


Figure adapted from 15-451/651 Lecture 14, October 21, 2015.



## Seidel Example: adding one constraint

- Suppose we know the best point for the first  $m - 1$  half-planes, then add one more half-plane.



## Seidel Example: adding one constraint

- Suppose we know the best point for the first  $m - 1$  half-planes, then add one more half-plane.
- If the old optimum satisfies it, nothing changes.



## Seidel Example: adding one constraint

- Suppose we know the best point for the first  $m - 1$  half-planes, then add one more half-plane.
- If the old optimum satisfies it, nothing changes.
- Otherwise, the new best point lies on the new boundary line. We scan that line using the earlier constraints.



## Adding one constraint

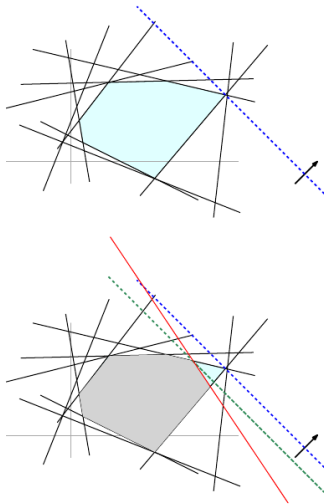


Figure adapted from 15-451/651 Lecture 14, October 21, 2015.



## Seidel example: a new constraint

- Maximize  $x$  inside a bounding box  $0 \leq x, y \leq 10$ , with constraints

$$C_1 : y \geq 2 \quad \text{and} \quad C_2 : y \leq 5.$$



## Seidel example: a new constraint

- Maximize  $x$  inside a bounding box  $0 \leq x, y \leq 10$ , with constraints

$$C_1 : y \geq 2 \quad \text{and} \quad C_2 : y \leq 5.$$

- After  $C_1$  and  $C_2$ ,  $(10, 2)$  is an optimal point.



## Seidel example: a new constraint

- Maximize  $x$  inside a bounding box  $0 \leq x, y \leq 10$ , with constraints

$$C_1 : y \geq 2 \quad \text{and} \quad C_2 : y \leq 5.$$

- After  $C_1$  and  $C_2$ ,  $(10, 2)$  is an optimal point.
- Add

$$C_3 : x + y \leq 8.$$

The old point violates it because  $10 + 2 > 8$ .



## Seidel example: a new constraint

- Maximize  $x$  inside a bounding box  $0 \leq x, y \leq 10$ , with constraints

$$C_1 : y \geq 2 \quad \text{and} \quad C_2 : y \leq 5.$$

- After  $C_1$  and  $C_2$ ,  $(10, 2)$  is an optimal point.
- Add

$$C_3 : x + y \leq 8.$$

The old point violates it because  $10 + 2 > 8$ .

- Therefore, the new best point lies somewhere on the boundary line  $x + y = 8$ .



## Seidel example: the one-dimensional problem

- On the new boundary,  $x = 8 - y$ . Thus maximizing  $x$  means making  $y$  as small as possible.



## Seidel example: the one-dimensional problem

- On the new boundary,  $x = 8 - y$ . Thus maximizing  $x$  means making  $y$  as small as possible.
- The earlier constraints give

$$2 \leq y \leq 5.$$



## Seidel example: the one-dimensional problem

- On the new boundary,  $x = 8 - y$ . Thus maximizing  $x$  means making  $y$  as small as possible.
- The earlier constraints give

$$2 \leq y \leq 5.$$

- This is the feasible part of the new boundary line. Its smallest value is  $y = 2$ , which gives

$$(x, y) = (6, 2).$$



## Seidel example: the one-dimensional problem

- On the new boundary,  $x = 8 - y$ . Thus maximizing  $x$  means making  $y$  as small as possible.
- The earlier constraints give

$$2 \leq y \leq 5.$$

- This is the feasible part of the new boundary line. Its smallest value is  $y = 2$ , which gives

$$(x, y) = (6, 2).$$

- $C_3$  finds the line by rechecking  $C_1$  and  $C_2$ .



## Why the two-dimensional algorithm is fast

- At step  $i$ , testing whether the current point satisfies the new half-plane takes  $O(1)$  time.



## Why the two-dimensional algorithm is fast

- At step  $i$ , testing whether the current point satisfies the new half-plane takes  $O(1)$  time.
- At most two boundary lines are needed to identify the final best point. In a random order, the  $i$ th line has probability at most  $2/i$  of changing that point.



## Why the two-dimensional algorithm is fast

- At step  $i$ , testing whether the current point satisfies the new half-plane takes  $O(1)$  time.
- At most two boundary lines are needed to identify the final best point. In a random order, the  $i$ th line has probability at most  $2/i$  of changing that point.
- If it changes the point, scanning the  $i - 1$  earlier constraints along its boundary line takes  $O(i)$  time.



## Why the two-dimensional algorithm is fast

- At step  $i$ , testing whether the current point satisfies the new half-plane takes  $O(1)$  time.
- At most two boundary lines are needed to identify the final best point. In a random order, the  $i$ th line has probability at most  $2/i$  of changing that point.
- If it changes the point, scanning the  $i - 1$  earlier constraints along its boundary line takes  $O(i)$  time.
- The expected extra work at step  $i$  is therefore at most  $\frac{2}{i} \cdot O(i) = O(1)$ .



## Seidel's running time

- There are  $n$  steps, and each costs expected  $O(1)$  time. Thus Seidel's two-dimensional algorithm takes expected  $O(n)$  time.



## Seidel's running time

- There are  $n$  steps, and each costs expected  $O(1)$  time. Thus Seidel's two-dimensional algorithm takes expected  $O(n)$  time.
- (More generally, in any fixed dimension  $d$ , the expected running time is  $O(d! n)$ . The factorial dependence is why this method is mainly used in low dimensions.)



## Section 3

### Integer Linear Programming



## From LP to integer linear programming

- An **Integer Linear Program (ILP)** adds integer restrictions to an LP.



## From LP to integer linear programming

- An **Integer Linear Program (ILP)** adds integer restrictions to an LP.

$$\text{LP: } \min c^T x \quad \text{subject to } Ax \leq b, \quad x \in \mathbb{R}^d,$$

$$\text{ILP: } \min c^T x \quad \text{subject to } Ax \leq b, \quad x \in \mathbb{Z}^d.$$



## From LP to integer linear programming

- An **Integer Linear Program (ILP)** adds integer restrictions to an LP.

$$\text{LP: } \min c^T x \quad \text{subject to } Ax \leq b, \quad x \in \mathbb{R}^d,$$

$$\text{ILP: } \min c^T x \quad \text{subject to } Ax \leq b, \quad x \in \mathbb{Z}^d.$$

- The inequalities still define the same region. The ILP only allows its integer points.



## From LP to integer linear programming

- An **Integer Linear Program (ILP)** adds integer restrictions to an LP.

$$\text{LP: } \min c^T x \quad \text{subject to } Ax \leq b, \quad x \in \mathbb{R}^d,$$

$$\text{ILP: } \min c^T x \quad \text{subject to } Ax \leq b, \quad x \in \mathbb{Z}^d.$$

- The inequalities still define the same region. The ILP only allows its integer points.
- LP has polynomial-time algorithms, but general ILP is NP-hard.



## An ILP example

- A workshop makes tables ( $x$ ) and chairs ( $y$ ). It has four units of each of two resources. A table uses  $(2, 1)$  units and a chair uses  $(1, 2)$  units.



## An ILP example

- A workshop makes tables ( $x$ ) and chairs ( $y$ ). It has four units of each of two resources. A table uses  $(2, 1)$  units and a chair uses  $(1, 2)$  units.
- The integer linear program is

$$\begin{array}{ll}\max & 3x + 2y \\ \text{subject to} & 2x + y \leq 4, \\ & x + 2y \leq 4, \\ & x, y \in \mathbb{Z}_{\geq 0}.\end{array}$$



## An ILP example

- A workshop makes tables ( $x$ ) and chairs ( $y$ ). It has four units of each of two resources. A table uses  $(2, 1)$  units and a chair uses  $(1, 2)$  units.
- The integer linear program is

$$\begin{array}{ll}\max & 3x + 2y \\ \text{subject to} & 2x + y \leq 4, \\ & x + 2y \leq 4, \\ & x, y \in \mathbb{Z}_{\geq 0}.\end{array}$$

- The LP *relaxation* (ignoring integer constraints) has  $x = y = \frac{4}{3}$  and value  $\frac{20}{3}$ . The ILP optimum is  $(x, y) = (2, 0)$  with value 6.



## How branch and bound works

- **Branch and bound [1960]** is an exact ILP algorithm. It explores integer cases while using LP bounds to skip cases that cannot win.



## How branch and bound works

- **Branch and bound [1960]** is an exact ILP algorithm. It explores integer cases while using LP bounds to skip cases that cannot win.
- First ignore the integer restrictions and solve the LP relaxation. Its objective value bounds every integer solution in that branch.



## How branch and bound works

- **Branch and bound [1960]** is an exact ILP algorithm. It explores integer cases while using LP bounds to skip cases that cannot win.
- First ignore the integer restrictions and solve the LP relaxation. Its objective value bounds every integer solution in that branch.
- If the LP solution is fractional, choose one fractional variable and split the search into two cases, such as  $x \leq \lfloor x^* \rfloor$  and  $x \geq \lceil x^* \rceil$ .



## How branch and bound works

- **Branch and bound [1960]** is an exact ILP algorithm. It explores integer cases while using LP bounds to skip cases that cannot win.
- First ignore the integer restrictions and solve the LP relaxation. Its objective value bounds every integer solution in that branch.
- If the LP solution is fractional, choose one fractional variable and split the search into two cases, such as  $x \leq \lfloor x^* \rfloor$  and  $x \geq \lceil x^* \rceil$ .
- Whenever the solver finds a valid integer solution, it stores its value as the incumbent.



## How branch and bound works

- **Branch and bound [1960]** is an exact ILP algorithm. It explores integer cases while using LP bounds to skip cases that cannot win.
- First ignore the integer restrictions and solve the LP relaxation. Its objective value bounds every integer solution in that branch.
- If the LP solution is fractional, choose one fractional variable and split the search into two cases, such as  $x \leq \lfloor x^* \rfloor$  and  $x \geq \lceil x^* \rceil$ .
- Whenever the solver finds a valid integer solution, it stores its value as the incumbent.
- Prune a branch when it is infeasible, already integer, or its LP bound cannot improve the incumbent. Once every branch is done or pruned, the incumbent is optimal.



## Branch and bound: search and prune

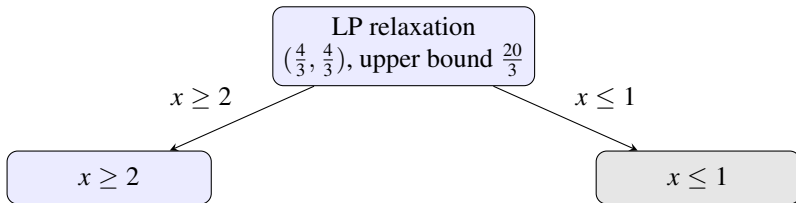
- The LP relaxation has upper bound  $\frac{20}{3}$  and fractional value  $x = \frac{4}{3}$ .

LP relaxation  
 $(\frac{4}{3}, \frac{4}{3})$ , upper bound  $\frac{20}{3}$



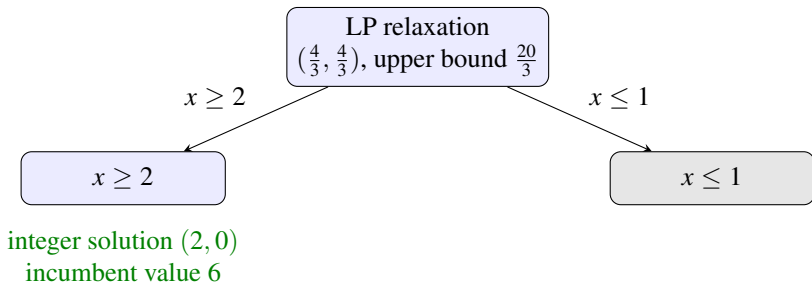
## Branch and bound: search and prune

- The LP relaxation has upper bound  $\frac{20}{3}$  and fractional value  $x = \frac{4}{3}$ .
- Branch into  $x \leq 1$  and  $x \geq 2$ .



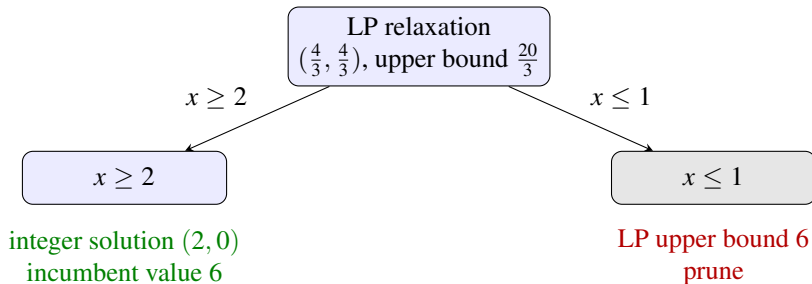
## Branch and bound: search and prune

- The LP relaxation has upper bound  $\frac{20}{3}$  and fractional value  $x = \frac{4}{3}$ .
- Branch into  $x \leq 1$  and  $x \geq 2$ .
- The  $x \geq 2$  branch gives  $(2, 0)$  with value 6. It becomes the incumbent.



## Branch and bound: search and prune

- The LP relaxation has upper bound  $\frac{20}{3}$  and fractional value  $x = \frac{4}{3}$ .
- Branch into  $x \leq 1$  and  $x \geq 2$ .
- The  $x \geq 2$  branch gives  $(2, 0)$  with value 6. It becomes the incumbent.
- The  $x \leq 1$  branch has LP upper bound 6, so it cannot improve the incumbent and is pruned.



## ILP solvers in Python

- In python, we just have to describe variables, linear constraints, and an objective. A solver performs the search.



## ILP solvers in Python

- In python, we just have to describe variables, linear constraints, and an objective. A solver performs the search.
- `scipy.optimize.milp` provides a direct interface to the HiGHS (High Performance Optimization Software) mixed-integer solver.



## ILP solvers in Python

- In python, we just have to describe variables, linear constraints, and an objective. A solver performs the search.
- `scipy.optimize.milp` provides a direct interface to the HiGHS (High Performance Optimization Software) mixed-integer solver.
- General ILP solvers combine presolve, LP relaxations, cutting planes (another algorithm), heuristics, and branch-and-bound.



## ILP solvers in Python

- In python, we just have to describe variables, linear constraints, and an objective. A solver performs the search.
- `scipy.optimize.milp` provides a direct interface to the HiGHS (High Performance Optimization Software) mixed-integer solver.
- General ILP solvers combine presolve, LP relaxations, cutting planes (another algorithm), heuristics, and branch-and-bound.
- When the solver reports optimal status, it has proved that no better feasible integer solution exists, subject to numerical tolerances.



Questions?

